



IMDAD AI

AI-Powered Ticket Management Agent

Complete Project Report and Mentor Presentation Documentation

Project Type	Role-based AI help desk and support ticket workflow system
Prepared For	Mentor review, viva, demo presentation, and project evaluation
Prepared By	Nida Tasneem
Current Status	Core user, engineer, admin, database, analytics, and demo reset milestones completed

Executive positioning

Imdad AI demonstrates how a modern support desk can reduce repetitive IT tickets by combining Knowledge Base search, AI troubleshooting, automatic ticket creation, role-based dashboards, and live Supabase-backed workflow tracking.

Table of Contents

1. Executive Summary	3
2. Problem Statement and Project Objectives	3
3. Completed Project Journey	4
4. Technology Stack.....	4
5. System Architecture	5
6. Role-Based Modules.....	6
7. AI Help and Ticket Pipeline	7
8. Supabase Database Design.....	8
9. UI and UX Development	9
10. Engineer Workflow and Resolution History	13
11. Admin Analytics and Reporting	14
12. Testing and Verification.....	15
13. Challenges Faced and Solutions.....	16
14. Current Completion Milestone	16
15. Future Scope and Presentation Notes	17
Appendix A: File and Database Reference	18

1. Executive Summary

Imdad AI is a role-based AI help desk application built to help users solve common technical problems quickly and create structured support tickets only when human support is needed. The system supports Users, Support Engineers, and Admins, each with a focused dashboard and permission-aware navigation.

The project started as an AI Ticket Management Agent concept and has evolved into a complete multi-page web application connected to Supabase. It now includes authentication, session management, AI help flow, Knowledge Base search, ticket creation, user ticket tracking, engineer ticket management, admin analytics, Knowledge Base management, and resolution history.

Why this project stands out

The application does not only collect tickets. It tries to prevent unnecessary tickets first through Knowledge Base matching and AI troubleshooting, then creates a clean, trackable ticket with priority, team assignment, AI suggestion, and support workflow metadata.

Area	Details
Project name	Imdad AI
Core idea	Smart AI-powered support ticket system
Primary users	Employees, support engineers, and administrators
Database status	Supabase schema repaired, demo data reset, and resolution history added
Current milestone	Modules through Reports and Analytics have been built and verified at prototype level

2. Problem Statement and Project Objectives

Problem Statement

In many organizations, users raise tickets for repeated issues such as VPN problems, password reset, email blocking, access denied, slow systems, printer issues, or application errors. Support engineers lose time answering the same issues manually, while admins have limited visibility into ticket trends, workload, and resolution performance.

Objectives

- Allow users to sign up, log in, describe technical issues, and receive instant help.
- Search the Knowledge Base before escalating to AI or human support.
- Automatically create support tickets when the user confirms that the issue is not resolved.
- Generate ticket ID, priority, team assignment, status, and timestamps automatically.
- Provide support engineers with ticket management, resolution notes, status updates, and history.
- Provide admins with user control, Knowledge Base management, ticket monitoring, and analytics.
- Store live data in Supabase so the prototype behaves like a real support system.

Scope

The current scope is a working full-stack prototype suitable for mentor demonstration. It includes a React front end, Express AI API, Supabase database, role-based workflow, and a responsive SaaS-style interface. Production deployment, advanced Supabase Auth policies, audit-grade security, and notification integrations are listed as future enhancements.

3. Completed Project Journey

Phase	Work Completed	Result
Concept and requirements	Defined User, Support Engineer, and Admin roles with AI help, ticketing, Knowledge Base, chat history, and analytics requirements.	Clear role-based product scope.
Initial web app	Built the React/Vite project and organized content into actual pages instead of placing everything on the home page.	Professional multi-page app structure.
Interface upgrade	Created a SaaS-style home page, role-based navbar, 3D assets, FAQ, footer, smooth transitions, and responsive layouts.	The app gained a clearer visual hierarchy and improved presentation quality.
Supabase connection	Connected the app to Supabase using environment variables and the supabase-js client.	Real database reads and writes became available.
Database repair	Fixed schema mismatches, removed old auth.users dependency, added password_hash, repaired ticket fields, and reset demo data.	Clean Supabase schema with stable demo records.
Engineer modules	Implemented engineer authentication, protected dashboard, open ticket management, update status, assign engineer, remarks, resolution notes, and resolution history.	Support workflow became operational.
Admin and analytics	Polished admin dashboard with KPIs, statistics, category/priority/status reporting, engineer performance, and Knowledge Base management.	Admin can monitor complete system activity.
Testing and verification	Verified build, login, ticket creation, engineer updates, resolution history, admin analytics, and Supabase refresh behavior.	Current milestone is functionally complete for demo use.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React 19, TypeScript, Vite	Fast single-page application with route-based views and reusable components.
Styling	CSS, custom design system, 3D SVG/PNG assets, lucide-react icons	Consistent SaaS-style interface, cards, dashboards, forms, tables, and visual hierarchy.

Layer	Technology	Purpose
Backend API	Node.js, Express	Provides `/api/troubleshoot` for AI troubleshooting requests.
AI	Ollama Llama 3.2 with fallback suggestions	Generates technical troubleshooting steps when no Knowledge Base match exists.
Database	Supabase Postgres	Stores users, tickets, Knowledge Base entries, chat history, and resolution history.
Auth prototype	Role-based app login with hashed password values and local session persistence	Controls redirection and dashboard access by role.
Verification	TypeScript build and browser testing	Checks UI, workflows, persistence, and realtime-like data refresh.

Important security note

The current project is a prototype. Supabase Row Level Security policies are permissive for demo speed. In production, the next step is to replace prototype policies with authenticated Supabase Auth rules and role-specific policies.

5. System Architecture

The system uses a client-first web architecture. React manages the interface, role navigation, forms, dashboards, and local session state. Supabase stores live application data. The Express server connects the AI Help module to Ollama Llama 3.2 and returns structured troubleshooting advice with a safe fallback when the local AI model is unavailable.

Component	Responsibility
React application	Displays pages, handles role-based navigation, validates inputs, calls Supabase, and renders dashboards.
Supabase client	Reads and writes users, tickets, Knowledge Base entries, chat history, and ticket resolution history.
Express API	Receives issue descriptions and requests AI-generated troubleshooting from Ollama.
Ollama Llama 3.2	Generates troubleshooting steps for custom issues that are not solved by the Knowledge Base.
Local session	Stores the current signed-in role and redirects users to the correct dashboard.

System Architecture

Role-based React app connected to Supabase, Express API, and Ollama Llama 3.2.

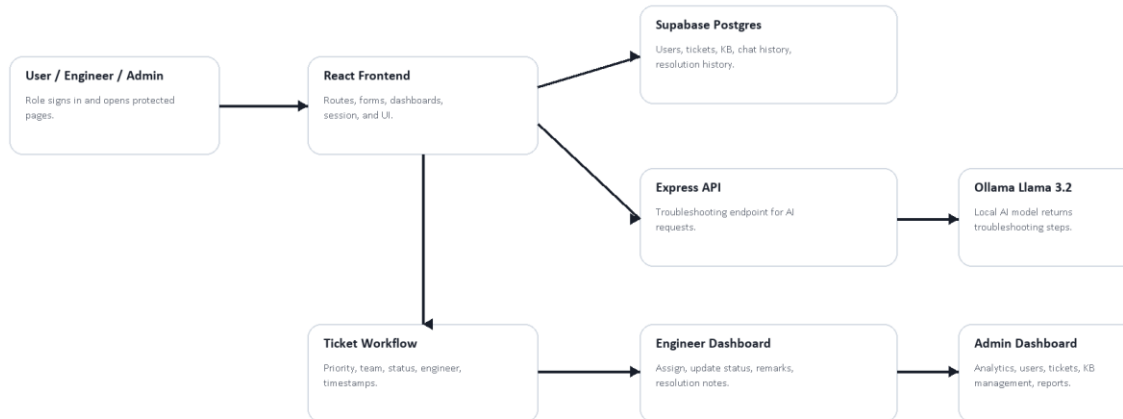


Figure 1. System architecture showing React, Supabase, Express API, Ollama, ticket workflow, and role dashboards.

Architecture Flow

1. User opens Imdad AI and selects the correct role.
2. After login, the app stores a session and redirects based on the role.
3. User enters an issue in AI Help.
4. System searches Supabase Knowledge Base by keywords.
5. If a solution exists, it is shown immediately.
6. If no match exists, the Express API asks Ollama Llama 3.2 for troubleshooting steps.
7. If resolved, chat history is saved.
8. If not resolved, a ticket is created and stored in Supabase.
9. Engineers update the ticket status, assignment, remarks, resolution notes, and history.
10. Admin monitors users, tickets, Knowledge Base, statistics, and reports.

6. Role-Based Modules

User Role

- Create account and log in.
- Use AI Help for troubleshooting.
- Receive Knowledge Base result or AI suggestion.
- Mark issue as resolved or proceed with human support.
- Create support tickets when needed.
- View only own tickets, own ticket details, own chat history, and resolution updates.

Support Engineer Role

- Log in to a protected engineer dashboard.
- View open, assigned, and available support tickets.

- Search and filter tickets by issue, status, priority, team, and recency.
- Review user issue details, AI suggestion, and ticket context.
- Assign tickets, update status, add remarks, add resolution notes, resolve, and close tickets.
- Track resolution history linked to each ticket.

Admin Role

- Log in to the admin dashboard.
- View users, support engineers, tickets, status reports, priority reports, category analysis, and engineer performance.
- Manage Knowledge Base entries with add, update, delete, and category selection.
- Monitor support activity, ticket volume, open workload, resolved tickets, and high-priority issues.

Role	Home after Login	Key Protected Pages
User	AI Help / user home	AI Help, Raise Ticket, My Tickets, Ticket Details, Chat History, Knowledge Base
Support Engineer	Engineer Analytics Dashboard	Engineer Home, Tickets, Ticket Details, Chat History, Knowledge Base
Admin	Admin Analytics Dashboard	Admin Home, Users, Tickets, KB Management

7. AI Help and Ticket Pipeline

The AI Help module is designed to reduce unnecessary ticket creation. Users first describe a technical problem. The system checks common Knowledge Base entries. Only when no matching Knowledge Base answer is available does the AI model generate troubleshooting steps.

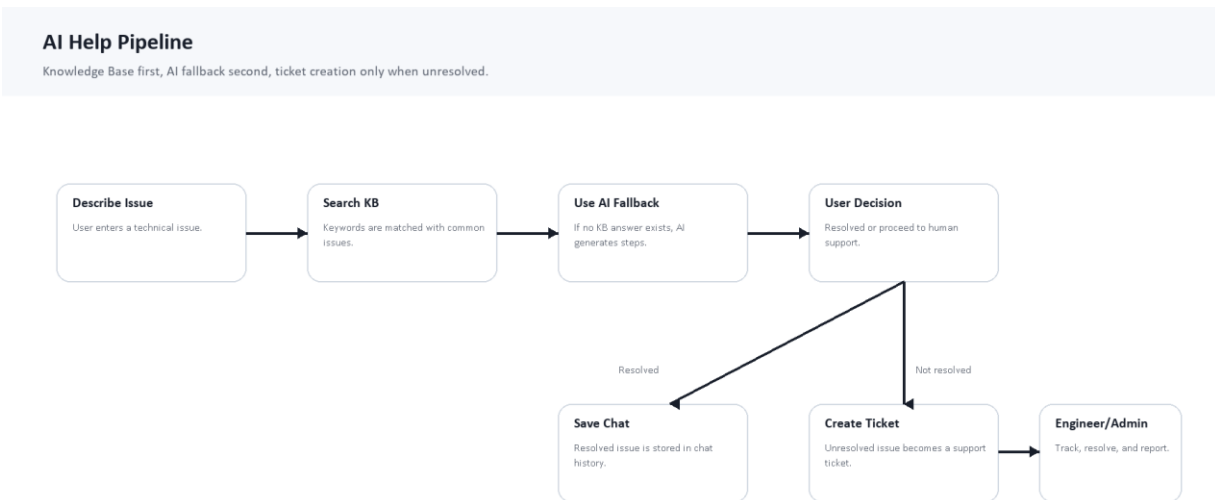


Figure 2. AI Help pipeline showing Knowledge Base search, AI fallback, user decision, chat history, ticket creation, and escalation.

Step	Action	Stored Data
1	User enters issue.	Issue text and signed-in user context.
2	Knowledge Base keyword search runs.	Matched entry source if available.
3	If no match exists, AI suggestion is generated.	AI response text and source value.
4	User selects Issue Resolved.	Chat history with result: Resolved without ticket.
5	User selects Human Support.	Ticket record plus chat history linked to ticket ID.
6	Ticket priority and team are inferred.	Priority, assigned team, status Open, timestamps.

Priority Rules

Detected Issue Type	Priority
Server down, system down, access denied, email blocked	High
Critical, security breach, data loss, ransomware	Critical
VPN, network, Wi-Fi, email, printer, software	Medium
Password reset or general low-impact issue	Low

Team Assignment Rules

Issue Keywords	Assigned Team
VPN, Network, Wi-Fi	Network Team
Email, Outlook, Mail	IT Support Team
Printer, Laptop, Hardware, Display	Hardware Team
Password, Login, Access, MFA	Access Management Team
Software, Application, Installation	Software Support Team
Unknown issue	General Support Team

8. Supabase Database Design

The live Supabase database has been repaired and reset using the project SQL script. The schema now supports application-owned users, ticket records, Knowledge Base entries, chat history, and engineer resolution history.

Table	Main Purpose	Important Fields
users	Stores user, support engineer, and admin accounts.	id, full_name, email, role, employee_id, department, password_hash, created_at
knowledge_base	Stores common IT issue solutions.	id, issue_name, keywords, solution, created_at, updated_at
tickets	Stores support tickets and lifecycle status.	ticket_id, user_id, user details, issue, AI suggestion, priority, assigned team, engineer, status, resolution, timestamps
chat_history	Stores every AI/Knowledge Base support interaction.	user details, issue, ai_response, source, result, related ticket_id, created_at
ticket_resolution_history	Stores engineer workflow updates.	ticket_id, engineer, previous_status, new_status, remarks, resolution_notes, created_at

Current Verified Demo Data

Data Area	Verified Status
Users	4 accounts: 2 users, 1 support engineer, 1 admin.
Knowledge Base	16 common IT issue entries.
Tickets	2 clean demo tickets after reset.
Chat History	2 demo AI/Knowledge Base interactions.
Resolution History	1 engineer update linked to TKT-0001.

Database repair completed

The earlier problem with an old auth.users foreign key and missing password_hash was fixed through the SQL repair script. The database repair script was executed and the repaired schema was verified successfully.

9. UI and UX Development

The project was redesigned from a single-page content dump into a structured multi-page SaaS-style application. Each page now has a focused purpose, clear hierarchy, proper spacing, role-specific navigation, and relevant visual assets.

Page / Area	UI Work Completed
Home	Clean landing page with navbar, hero, feature cards, simple workflow, FAQ, and footer.
Login and Signup	Role-based forms, support engineer department list, session redirection, and logout flow.
AI Help	Aligned issue input, result panel, four action buttons, Knowledge Base answer, AI suggestion, and 3D visual.

Page / Area	UI Work Completed
Raise Ticket	Submit Support Request heading, placeholders, clean form, latest ticket preview, no-ticket layout, and View All Tickets action.
My Tickets	Paginated user ticket list with ticket ID, issue, priority, status, and View Detail button.
Knowledge Base	Search bar, quick review issues, list layout, aligned issue type/category labels, and no keyword clutter.
Engineer Dashboard	Analytics-style dashboard, KPI cards, ticket table, search/filter/sort, pagination, and ticket management.
Admin Dashboard	Operational control dashboard, analytics cards, reports, users list, engineer list, and Knowledge Base management.

- Project name changed to Imdad AI.
- Custom logo and watermark-style brand mark added across the project.
- Theme refined using a consistent SaaS visual direction inspired by the NEFA reference, adapted to the Imdad AI support domain.
- Scroll transitions were added and later smoothed to reduce UI lag.
- Navbar active highlighting was added so users know which page they are on.

Implementation Evidence Screenshots

The following role-based collages show representative screens from the implemented application. They connect the documented workflow to visible application output.

User Pages and Activity

AI Help, ticket creation, and personal ticket tracking.

AI Help

AI HELP

Quick Fix Before Ticket

Enter your issue. The system searches the Knowledge Base, then applies AI if no solution is found.

ISSUE INPUT

Describe the problem

Type your issue here (e.g., VPN not connecting after MFA).

[Get Knowledge Base Answer](#)[AI Suggestion](#)

KNOWLEDGE BASE RESULT / AI RESPONSE

Run a triage check to see a Knowledge Base match or AI troubleshooting response.

Raise Ticket

RAISE TICKET

Submit Support Request

Review the AI context, priority, and assigned team before creating the ticket.

Issue Title

For eg.: Laptop screen flickering

Issue description

For eg.: My laptop screen keeps flickering during work, especially when I join Teams meetings or connect to an external monitor.

AI suggestion

VIEW ADDED TICKET

TXT-0002

Laptop slow after update

Laptop is slow after today's update and Teams calls freeze.

Priority

Medium

Team

Hardware Team

Status

Open

[View All Tickets](#)[View Details](#)

My Tickets

MY TICKETS

Track your support requests

View and track all tickets linked to your account.

Ticket ID	Issue Title	Priority	Status	Action
TXT-0002	Laptop slow after update	Medium	Open	View Detail

Showing 1-1 of 1

Figure 3. User pages: AI Help, Raise Ticket, and My Tickets.

Imdad AI Complete Project Report | Nida Tasneem | Page 11 of 18

Support Engineer Pages and Activity

Engineer dashboard, ticket queue, and ticket context review.

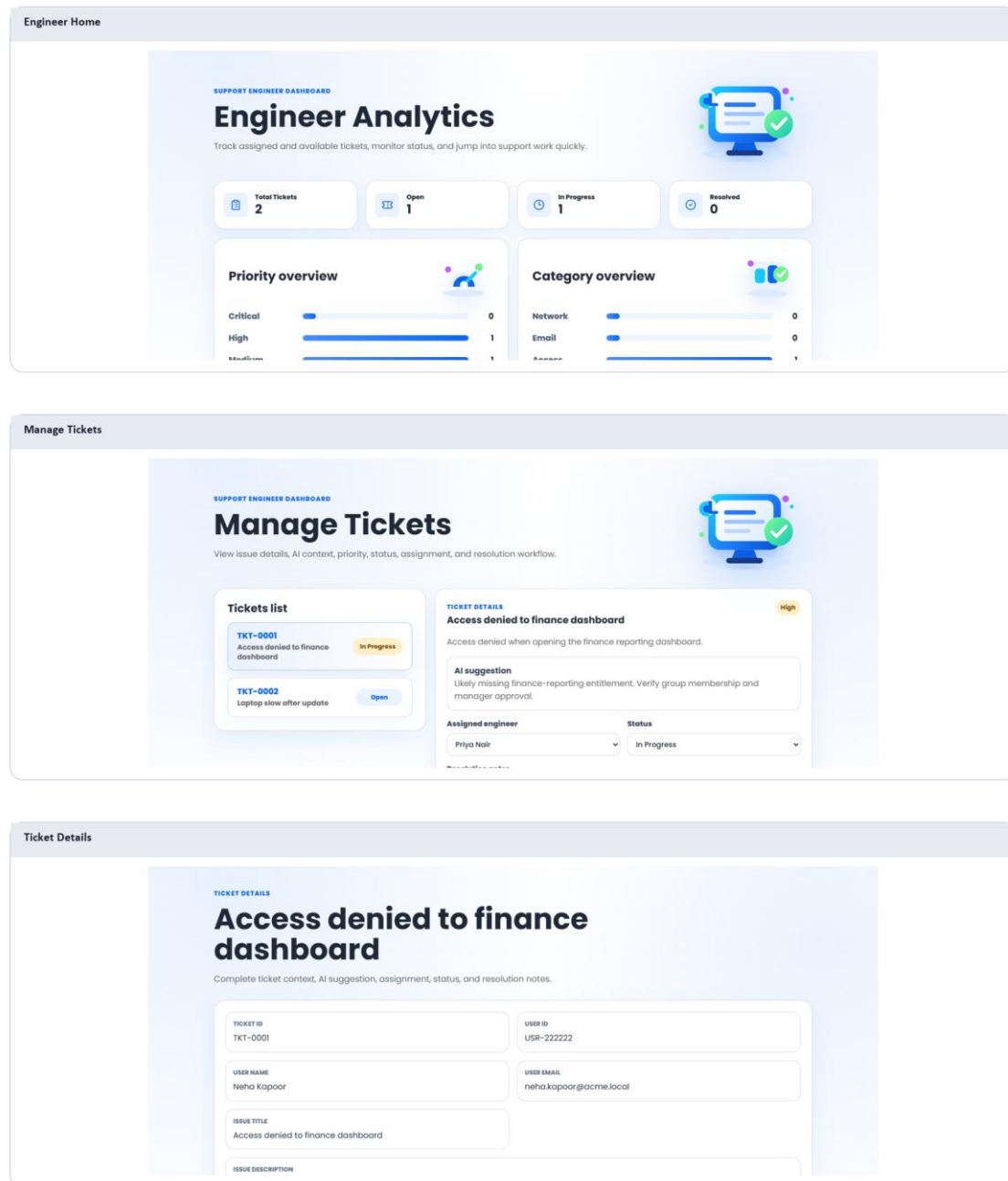


Figure 4. Support Engineer pages: dashboard, ticket management, and ticket detail review.

Admin Pages and Activity

Analytics, Knowledge Base management, and ticket monitoring.

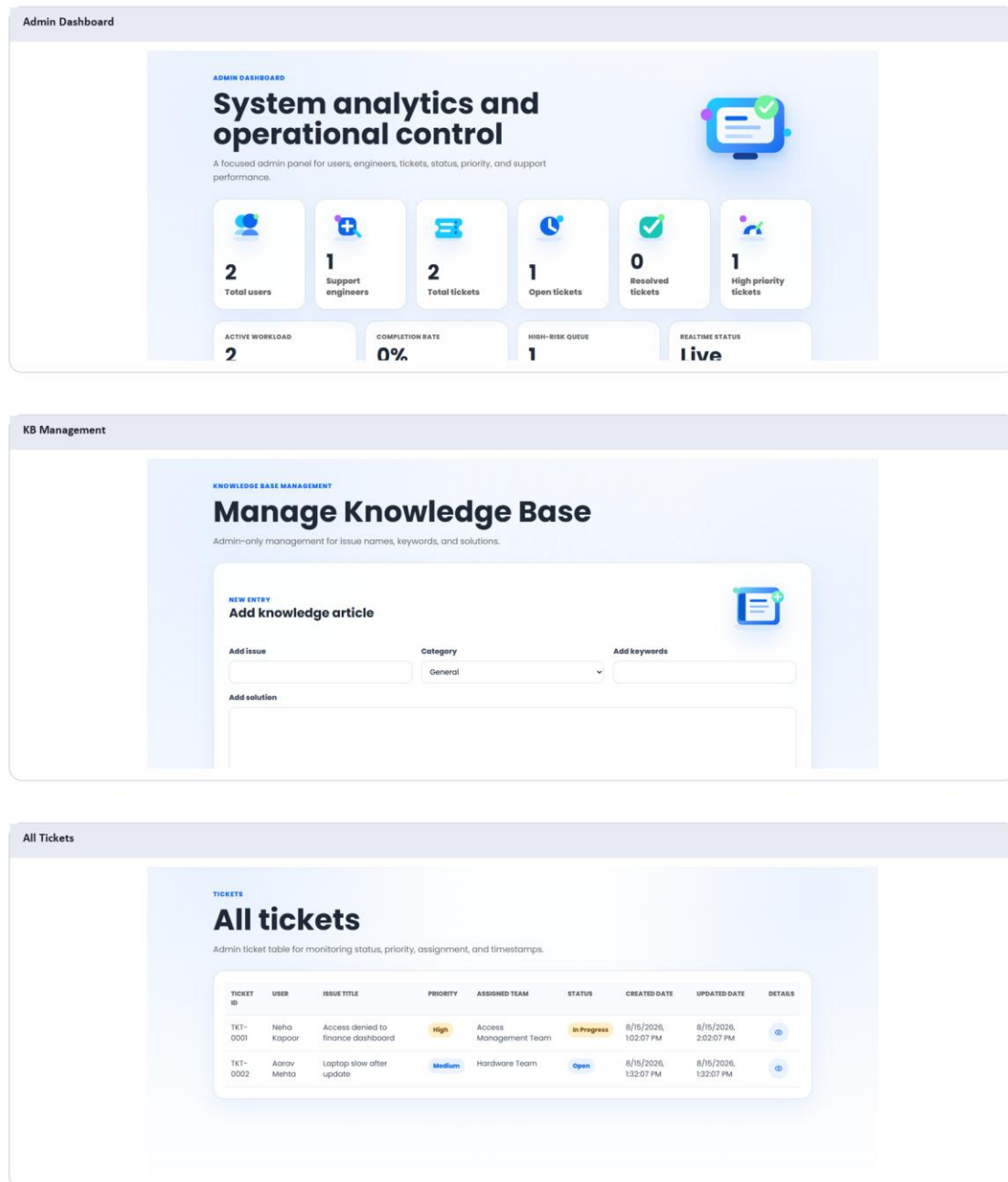


Figure 5. Admin pages: analytics dashboard, Knowledge Base management, and ticket monitoring.

10. Engineer Workflow and Resolution History

The Support Engineer module now goes beyond viewing tickets. It supports ticket operations, update tracking, and reporting behavior that resembles a real service desk workflow.

Engineer Feature	Implementation Status
Login system	Available with role-based support engineer login.
Session management	Current session persists in browser storage and controls navigation.
Protected dashboard	Engineer pages require the correct role.
KPI cards	Dashboard shows total, open, in progress, resolved, and workload indicators.
Ticket statistics	Status, priority, and category overview included.
Open ticket management	Engineer can view, search, filter, and open ticket details.
AI review	Ticket detail includes AI suggestion and issue context.
Resolution workflow	Engineer can assign, update status, add remarks, add resolution notes, resolve, and close tickets.
Resolution history	Every engineer update is saved in the ticket_resolution_history table.

11. Admin Analytics and Reporting

The Admin dashboard has been shaped as the main analytics area, so the separate Analytics navigation item was removed. Admin Home now provides system control, ticket visibility, and reporting in one place.

Report Area	What It Shows
KPI cards	Total users, support engineers, total tickets, open tickets, resolved tickets, high-priority tickets.
Tickets by status	Open, In Progress, Resolved, and Closed ticket distribution.
Tickets by priority	Critical, High, Medium, and Low ticket distribution.
Category-wise analysis	Ticket breakdown by support area such as Access, Hardware, Network, Software, and Email.
Resolution trends	Resolved and closed ticket movement based on current ticket status and history.
Engineer performance	Assigned workload and completed work per engineer.
Ticket analytics table	Searchable, filterable, sortable, and paginated ticket report for admin review.

Realtime data behavior

The app subscribes to Supabase data changes and also uses a timed refresh fallback, so dashboards remain current even if realtime publication behavior is delayed in the demo environment.

12. Testing and Verification

Testing focused on the full support lifecycle: login, role redirection, Knowledge Base search, AI fallback, ticket creation, ticket visibility, engineer workflow, resolution history, admin analytics, and Supabase data persistence.

Test Case	Expected Result	Actual Result	Status
Build verification	TypeScript and Vite build should complete without errors.	`npm run build` completed successfully after the latest implementation changes.	PASS
User login	User should access only user-specific pages after login.	User role opened AI Help, Raise Ticket, My Tickets, and Knowledge Base views.	PASS
Support engineer login	Engineer should access protected engineer dashboard and ticket tools.	Support Engineer role opened dashboard, ticket queue, and ticket detail workflow.	PASS
Admin login	Admin should access system monitoring and management pages.	Admin role opened analytics, users, tickets, and Knowledge Base management views.	PASS
Ticket creation	Unresolved issue should create a structured ticket with generated metadata.	Ticket was created with ticket ID, priority, team, status, timestamps, and AI context.	PASS
My Tickets access	User should see only tickets linked to the signed-in account.	User ticket list displayed account-linked tickets with pagination and detail action.	PASS
Engineer update	Engineer should update assignment, status, remarks, and resolution notes.	Engineer workflow saved assignment/status updates and resolution notes.	PASS
Resolution history	Engineer updates should be recorded in resolution history.	Updates were saved in `ticket_resolution_history` and shown on ticket details.	PASS
Supabase reset	Demo reset should recreate clean users, KB entries, tickets, chat history, and history rows.	Repair/reset script produced the expected clean demo dataset.	PASS
Live data check	Repaired users table should support insert/delete with password_hash.	Supabase insert/delete verification succeeded on the repaired schema.	PASS

Demo Commands

Command	Purpose
npm run dev	Starts the Vite frontend at 127.0.0.1:5173.
npm run server	Starts the Express AI API at 127.0.0.1:8787.
npm run build	Checks TypeScript and produces a production build.

13. Challenges Faced and Solutions

Challenge	Solution Applied
Supabase schema mismatch	Updated SQL to match actual table columns such as issue_name and integer KB IDs.
Old auth.users foreign key	Repaired users table to use project-owned UUID accounts instead of depending on deleted auth records.
Missing password_hash	Added password_hash column and updated account persistence logic.
Ticket ID compatibility	Updated generator to support existing legacy TCK IDs and new TKT IDs.
AI response was too generic for custom issues	Improved fallback troubleshooting logic for slow systems, network, VPN, email, and general issues.
Single-page content overload	Restructured into proper routes and role-specific pages.
UI alignment issues	Iteratively refined navbar, AI Help, Raise Ticket, My Tickets, Knowledge Base, Engineer, Admin, and footer layouts.
Realtime reliability	Added refresh fallback alongside Supabase realtime subscription.

14. Current Completion Milestone

The project has reached the stage where the main end-to-end prototype can be demonstrated to a mentor: users can request help, tickets are created and stored, engineers can manage tickets, admins can monitor analytics, and Supabase contains clean live demo data.

Module	Status	Notes
User Authentication and Dashboard	Completed	Signup, login, session, logout, role redirection, and user pages are implemented.
AI Help Module	Completed	Knowledge Base search, AI suggestion, resolved flow, and ticket escalation are implemented.
Ticket Creation and User Tracking	Completed	Automatic ticket ID, priority, team, status, My Tickets, details, and chat history are implemented.
Knowledge Base	Completed	User-facing search and admin management are implemented.
Engineer Authentication	Completed	Engineer login, protected dashboard, session, and logout are available.
Engineer Dashboard	Completed	KPIs, ticket stats, recent activity, priority/category overview are available.
Open Ticket Management	Completed	Search, filters, detail page, AI review, and recommendation review are available.

Module	Status	Notes
Resolution Workflow	Completed	Remarks, notes, status update, assignment, close ticket, and resolution history are available.
Reports and Analytics	Completed	Admin/engineer analytics, category, priority, trend, and performance metrics are available.
Production hardening	Pending	Needs Supabase Auth, stricter RLS, deployment, notifications, and audit logging.

15. Future Scope and Presentation Notes

Recommended Future Enhancements

- Replace prototype role login with full Supabase Auth and secure role claims.
- Add strict Row Level Security policies so users can only read their own rows at the database level.
- Deploy frontend and backend to a cloud platform.
- Add email or Teams notifications when tickets are created, assigned, resolved, or closed.
- Add SLA timers, escalation rules, and overdue ticket alerts.
- Add file attachments for screenshots and error logs.
- Add downloadable reports for admin analytics.
- Add audit logging for all sensitive support operations.

How to Present the Project

Step	Presentation Action
1	Start with the problem: repeated IT issues waste support time and delay users.
2	Introduce Imdad AI as the solution: first self-help, then AI, then ticket escalation.
3	Show the Home page and role-based navbar.
4	Log in as a User and demonstrate AI Help, Knowledge Base result, unresolved issue, and ticket creation.
5	Open My Tickets and Ticket Details to show user-side tracking.
6	Log in as Support Engineer and show dashboard, open tickets, AI suggestion review, status update, remarks, and resolution history.
7	Log in as Admin and show analytics, users, engineers, tickets, and Knowledge Base management.
8	End with the database: show Supabase tables proving the system stores real records.

Conclusion

Imdad AI is a complete role-based AI support desk prototype with real database persistence, structured escalation, engineer workflow, and admin reporting. It demonstrates both product thinking and technical implementation from UI to database.

Appendix A: File and Database Reference

Project Area	Important Files
Frontend app	src/App.tsx, src/styles.css, src/main.tsx
Supabase client and store	src/supabaseClient.ts, src/supabaseStore.ts
AI backend	server/index.js
Database scripts	supabase/schema.sql, supabase/fix_schema_and_reset_demo.sql
Brand and visuals	public/brand, public/visuals
Documentation	docs folder

Clean Demo Records

Record Type	Examples
Users	Aarav Mehta, Neha Kapoor, Priya Nair, Maya Rao
Tickets	TKT-0001 Access denied to finance dashboard; TKT-0002 Laptop slow after update
Knowledge Base topics	VPN, Wi-Fi, email, password reset, access denied, slow system, printer, software, server down, file missing
Resolution history	TKT-0001 update from Open to In Progress by Priya Nair